Improving Counting Sort Algorithm Via Data Locality

Java implementation and benchmark replication using modified quicksort preprocessing

Robbie Devilly Thehan Arunadewa Dermot Yang Li Weihao

Algorithms Module | UCD School of Computer Science | April 2026

Context & Motivation

Why this project matters

- Counting sort is theoretically O(n + r), but practical runtime changes with cache and memory-access behaviour.
- When values are already close together or sorted, updates to count[] and output[] tend to be cheaper and more predictable.
- Randomly scattered values cause irregular jumps, so the same algorithm can run much slower in practice.

Core question
Can a short preprocessing phase reorganize the input so counting sort behaves more like the sorted-input case?

What changes in the proposed method?

- Use modified quicksort as a preprocessing phase before counting sort.
- Partition the array into value-local blocks instead of treating the whole input as one scattered region.
- Stop recurring when a block satisfies $n_i + r_i \leq C$ (threshold ≈ 1000), then run counting sort on each block.

Method Walkthrough

The preprocessing stage is not a full replacement for counting sort — it restructures the input so counting sort works on smaller, more cache-friendly partitions.

1 Input → **2 Preprocess** → **3 Count by block** → **4 Output**

Input
Start with an array whose values may be scattered across a wide range.

Preprocess
Modified quicksort uses median-of-three partitioning to form value-local blocks.

Count by block
Counting sort runs on smaller blocks where count[] and output[] cover a shorter local region.

Output
Because partitions are already value-ordered, the concatenated result is globally sorted.

Why this can help
The extra quicksort work only pays off when the locality benefit is large enough. That is why the paper studies several different n and r settings.

Cache Miss Analysis

Classic Counting Sort

Cache misses (random):
$3n + 4n/L + 2r/L + 6$

The 3n term dominates — each access to count[] or output[] can cause a cache miss.

With Preprocessing

Improvement (n ≈ r):
$3n - n/L - 2r/L - 3i$

Small partitions fit in L1 cache, eliminating conflict and capacity misses.

Key Findings

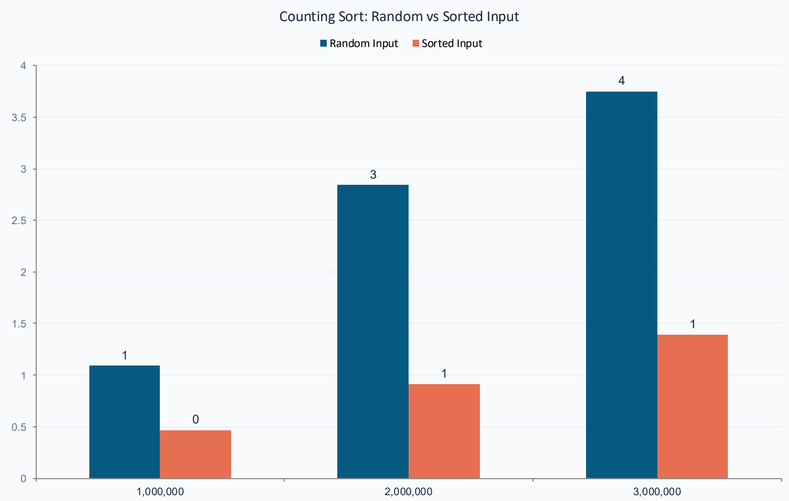
What the three benchmarks show

- Benchmark 1 compares random input with already sorted input to isolate the locality effect in plain counting sort.
- Benchmark 2 compares counting sort with and without modified-quicksort preprocessing for a large fixed range r = 1,000,000.
- Benchmark 3 compares classic quicksort, quicksort + insertion sort, and quicksort + counting sort when n = r.

Patterns visible across the benchmarks
Sorted/local data is consistently faster than random input. Preprocessing has visible overhead for small n, but the quicksort + counting sort variant becomes strongest in the large balanced n = r case.

Takeaway
Cache-aware rearrangement can change real execution time substantially, even when asymptotic complexity stays the same.

Benchmark 1: Random vs Sorted Input



n = r	Random input (ms)	Sorted input (ms)
1,000,000	1.09	0.467
2,000,000	2.84	0.910
3,000,000	3.75	1.390

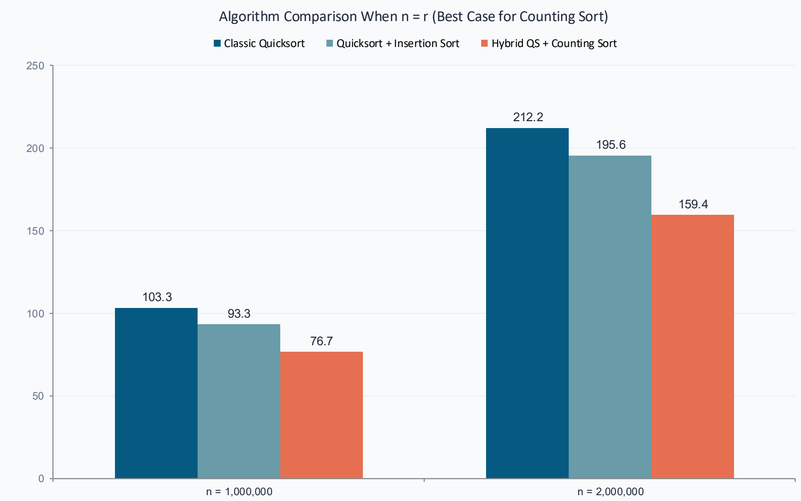
Result summary
Sorted input stays ahead at all three sizes: about 2.3× faster at 1M, 3.1× at 2M, and 2.7× at 3M. This strongly supports the locality explanation — when values are already ordered, auxiliary array accesses remain cache-friendly.

Benchmark 2: With vs Without Preprocessing

n	r	T_1 with prep (ms)	T_2 classic (ms)
1,000	1,000,000	0.14	0.90
2,000	1,000,000	0.22	1.23
3,000	1,000,000	0.63	0.94

Result summary
Preprocessing is expensive for the smallest case, but the locality-aware version stays competitive and clearly improves as n grows within the fixed large-range setting. The r >> n scenario is the worst case for counting sort, yet preprocessing still helps.

Benchmark 3: Quicksort Variant Comparison



n = r	Classic QS	QS + Insertion	Hybrid QS + Counting
1,000,000	103.3	93.3	76.7
2,000,000	212.2	195.6	159.4

Result summary
When n = r and the input is large, the quicksort + counting sort hybrid clearly beats both pure quicksort and the insertion-sort variant. The hybrid achieves a 25% speedup over classic quicksort at 1M inputs and similar gains at 2M.

Mahmud, S., Haque, S.A. and Choudhury, N. (2022). Improving Counting Sort Algorithm via Data Locality. ACMSE 2022.